Iterative Time Complexity

1. **Past Question -**
   CSE221_SolutionOfPastQuestions_TimeComplexity [Stude…

2. **Open DSA -** 4.6. Calculating Program Running Time — CS2 Software Design & Data Structures

3. **Princeton -** COS226 A-nalysis of Algorithms: Practice Examples Spring ‘20

4. **UNSW -** COMP 9024, Homework Solution, 11s2, Class 1

5. **UMBC -** CMSC 441: Homework #1 Solutions

6. **NUS -** Tutorial 9 – Analysis of Algorithms 1. Big-O Complexity

7. **HFN -** 1.3 time complexity Additional Solution and prep.pdf

8. **Examples shown in class** -
   Iterative Time Complexity Examples
9. **ANK** - Practice sheet time complexity

Previous Midterm Questions

1.

a. **Calculate** the time complexity of the following function:
CO7

```
worstCase(n):
        int i, j, k, a, b, sum
        for ( i = 0; i < n; i = i + 3)
                for ( j = n; j >= 1; j = j / 5)
                        for ( k = 1; k <= n; k = k * 5)
                                sum = a + b
```

2.

a.**[CO7]Calculate** the time complexity of the following function

Finding_Worst_Case(n):

```
int i,j,k,m,multi,a,b,c
for( i = n; i >= 1; i = i / 7 ){
      for( j = 1; j <= n; j = j + 3 ) {
            for( k=1; k<=40 ; k=k+1){
                  multi=a*b
             }
            for( m=n ; m>=1 ; m=m-5 ){
                  multi=multi*c
            }
       }
 }
```

3.

a. [**CO7**]**Calculate** the time complexity of the following function

Finding_Worst_Case(n):

```
int i,j,k,m,multi,a,b,c
for( i = n; i >= 1; i = i - 4 ){
        for( j = 1; j <= n; j = j * 3 ) {
            for( k=1; k<=20 ; k=k+1){
                multi=a*b
            }
            for( m=n ; m>=1 ; m=m/5 ){
                multi=multi*c
            }
        }
 }
```

4.

a. CO7 **Calculate** the time complexity of the following function:

```
worstCase(n):
        int i, j, k, a, b, sum
        for ( i = 1; i < n; i = i + 3)
                for ( j = n; j >= 1; j = j - 2)
                        for ( k = n; k >= 1; k = k / 4)
                                sum = a + b
```

5.

1	a. CO3	In the primary scholarship exam in Bangladesh, four lakh (n=4,00,000) students take part but only the top **50** students are given an award. **Write** the asymptotic time complexity to give the awards. Assume that each award is given in a constant time.	02
	b. CO1	Write the asymptotic time complexity of the following function.	04

```
1. def contains_duplicates(elements):
2.     for outer in range(len(elements)):
3.         for inner in range(len(elements)):
4.             if outer == inner:
5.                 continue
6.
7.             if elements[outer] == elements[inner]:
8.                 return True
9.
10.   return False
```

6.

1	a. CO3	In the primary scholarship exam in Bangladesh, two lakh (n=2,00,000) students take part but only the top **25** students are given an award. **Write** the asymptotic time complexity to give the awards. Assume that each award is given in a constant time.	02
	b. CO1	Write the asymptotic time complexity of the following function.	04

```
1. def cumulative_sum(elem):
2.     for outer in range(len(elem)):
3.         for inner in range(outer+1,len(elem)):
4.             elem[outer]= elem[outer] + elem[inner]
5.
6.     return elem
```

7.

1	a. CO2	**Find** the time-complexity of the following task in terms of number of students.	03

You are given a student attendance sheet. Each student has a unique integer ID. You have to count the number of students having an even number as ID. The list is sorted but the IDs are not necessarily consecutive. So you check each ID one by one.

	b. CO2	**Write** the asymptotic time complexity of the following code snippet. Show your works/reasoning.	02

```
1. for i in range (1,n)
2.     j= 1
3.     while j < i*i
4.         j= j+1
```

8.

1	a. CO2	**Find** the time-complexity of the following task in terms of number of students.	03

You are given a student attendance sheet. Each student has a unique integer ID. You have to count the number of students having an ID which is divisible by 3. The list is sorted but the IDs are not necessarily consecutive. So you check each ID one by one.

	b. CO2	**Write** the asymptotic time complexity of the following code snippet. Show your works/reasoning.	02

```
1. for i in range (1,n)
2.     j= 1
3.     while j*j < i
4.         j= j+1
```

9.

1.	**a.** **CO2**	Explain the time complexity of the following code snippet in regards of the Big-O notation: 1. `for (i=0; i<n; i+=4) {` 2. `for (j=1; j<n; j*=2) {` 3. `for (k=0; k<30; k++) {` 4. `print("Am I still not 30?!!");` 5. `}` 6. `print("Why, God, why? We had a Deal!");` 7. `for (m=n; m>0; m-=2) {` 8. `print("Could you BE more dramatic?");` 9. `}` 10. `}` 11. `}`	**04**
	b. **CO2**	Consider the following functions. $f_1(n) = (\log n)^{2023}$ $f_2(n) = n^2 \log_n(n^n)$ $f_3(n) = n^3 + 7n^2$ $f_4(n) = 2.023^n$ $f_5(n) = n \log n$ $f_6(n) = n * \sqrt[3]{n^2}$ Now do the followings: a. Write a correct asymptotic upper bound for each of the above. b. Sort the functions in ascending order of their growth rate, assuming n is significantly large. Just write the sorted order, no need to show any simulation.	**03** **03**

10.

a. **CO2** Explain the time complexity of the following code snippet in regards of the Big-O notation: **04**

```
1. for (i=0; i<n; i+=4) {
2.       for (j=1; j<n; j*=2) {
3.             for (k=0; k<20; k++) {
4.                   print("Am I still not 30?!!");
5.             }
6.             print("Why, God, why? We had a Deal!");
7.             for (m=n; m>0; m-=4) {
8.                   print("Could you BE more dramatic?");
9.             }
10.      }
11. }
```

b. **CO2** Consider the following functions.

$f_1(n) = (\log n)^{2000}$

$f_2(n) = n^3 \log_n(n^n)$

$f_3(n) = n^3 + 7n^2$

$f_4(n) = 4^n$

$f_5(n) = n \log n$

$f_6(n) = n * \sqrt{n}$

Now do the followings:

a. Write a correct asymptotic upper bound for each of the above. **03**

b. Sort the functions in ascending order of their growth rate, assuming n is significantly large. Just write the sorted order, no need to show any simulation. **03**

11.

a. **CO2** Consider the following functions. **03**

$f_1(n) = log\,(n!)$

$f_2(n) = n^2\, log_n(n^n)$

$f_3(n) = n^4 + 10n^2$

$f_4(n) = 17.13^n$

$f_5(n) = n\, log\, n$

$f_6(n) = n * \sqrt[5]{n^2}$

Write a correct asymptotic upper bound for each of the above and sort the functions in ascending order of their growth rate.

b. **CO2** **Write** the asymptotic time complexity of the following code snippet. Show your works/reasoning. **03**

```
1. for i in range (1,n)
2.     j= 1
3.     while j < i*i
4.         j= j-1
```

12.

a. **CO2** Consider the following functions. **03**

$f_1(n) = log\,(n!)$

$f_2(n) = n * \sqrt[4]{n^2}$

$f_3(n) = n^3 + 10n^2$

$f_4(n) = 17.13^n$

$f_5(n) = n\, log\, n$

$f_6(n) = n^2\, log_n(n^n)$

Write a correct asymptotic upper bound for each of the above and sort the functions in ascending order of their growth rate.

b. **CO2** **Write** the asymptotic time complexity of the following code snippet. Show your works/reasoning. **03**

```
1. for i in range (1,n)
2.     j= 1
3.     while j < i*i
4.         j= j+1
```

13.

For each of these pairs of functions $(f(n), g(n))$, identify whether $f(n) = O(g(n))$ or $f(n) = \square(g(n))$. **03**

Show your calculation logic in the script and **Tick only the most specific one** of the two options in each case in the question paper. ***Marks for this question depend on the calculation shown in your scripts***.

$f(n)$	$g(n)$	$f(n) = O(g(n))$	$f(n) = \square(g(n))$
3^n	$2^n . n^4$	☐	☐
$(\sqrt{n} + n)(30\sqrt{n})$	n^2	☐	☐

14.

a. **CO2** Analyze the following code snippet and **express** the time complexity using the Big O-notation. You must mention the time complexity for each loop. **04**

```
1. int b = 0;
2. for (int i = n; i > 0; i -= 2) {
3.     for (int j = n; j > 0; j--) {
4.         b += i * j;
5.     }
6. }
7.
8. for (int k = 1; k < n; k *= 4) {
9.     for (int m = 0; m < n; m += 3) {
10.          if (m == 10) {
11.              break;
12.          }
13.          b += k + m;
14.      }
15.  }
16.  return b;
```

15.

a. CO2 Analyze the following code snippet and **express** the time complexity using the Big O-notation. You must mention the time complexity for each loop. **04**

```
1. int b = 0;
2. for (int i = n; i > 0; i -= 3) {
3.     for (int j = 0; j < n; j += 3) {
4.         if (j == 50) {
5.             break;
6.         }
7.         b += i + j;
8.     }
9. }
10.
11.  for (int k = 1; k < n; k *= 3) {
12.      for (int m = n; m > n; m /= 5) {
13.          b += k + m;
14.      }
15.  }
16.  return b;
```

16.

1 a. CO2 Why do we ignore constants and lower-order terms when expressing the time complexity of an algorithm? Briefly **explain** in your own words. **01**

b. CO2 Is $n * \sqrt{n}$ asymptotically faster than $n * log_2(n)$? **Justify** with proper logic. **02**

c. CO2 **Write** the time complexity of the following code snippet. **03**

```
public static void func(int[] arr) {
    int n = arr.length;
    int k = 0;
    for (int i = 0; i < n; i++)
        k += arr[i];

    for (int i = 1; i < n; i++)
        for (int j = 1; j <= arr[i]; j++)
            System.out.print(i);
}
```